

# CH 1 Error & Approximations

## Truncation Errors

Truncation errors occur when an infinite series or iterative process is approximated by stopping at a finite number of terms. These errors commonly arise in:

- **Taylor Series Approximations:** Higher-order terms are ignored.
- **Numerical Differentiation and Integration:** Only a finite number of steps are considered.
- **Finite Difference Methods:** Discretization of continuous functions.

### Example:

Approximating  $e^x$  using the Taylor series:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

If we truncate after the second-order term, the truncation error is the sum of the ignored terms.

## Rounding Errors

Rounding errors occur due to the finite precision of digital computers. They arise when:

- A number cannot be exactly represented in the machine's memory.
- Computations involve floating-point arithmetic, leading to small deviations.

**Example:** In a system that stores only four decimal places, representing  $\pi=3.141592653$  as 3.1416 introduces a rounding error.

## Fixed-Point Arithmetic

- Represents numbers with a fixed number of decimal places.
- Suitable for financial and monetary calculations where precision is crucial.
- Limited in range since it cannot represent very large or very small numbers efficiently.

## Floating-Point Arithmetic

- Uses a scientific notation format:  $N=M \times B^E$ , where M is the mantissa, B is the base (usually 2 in binary systems), and E is the exponent.
- Allows representation of a much larger range of values.
- Used in scientific and engineering computations.
- **Example:** The number **0.0000345** can be written in floating-point as  $3.45 \times 10^{-5}$ .

## Propagation of Errors

Error propagation refers to how errors accumulate and amplify through a sequence of computations. There are two main types:

1. **Additive Error Propagation:** If numbers with errors are added or subtracted, the total error is the sum of the absolute errors. **Example:**

If  $x = 5.2 \pm 0.1$  and  $y = 3.1 \pm 0.05$ , then:

$$(x + y) = (5.2 + 3.1) \pm (0.1 + 0.05) = 8.3 \pm 0.15$$

- 2. Multiplicative Error Propagation:** When multiplying or dividing values, relative errors add up. **Example:**

If  $x = 5.2 \pm 0.1$  and  $y = 3.1 \pm 0.05$ , then the relative error in multiplication is:

$$\frac{\Delta z}{z} = \frac{\Delta x}{x} + \frac{\Delta y}{y}$$

## Significant Figures

Significant figures determine the precision of a numerical value.

- The number of **reliably known digits** in a number.
- Includes all digits except **leading zeroes**.

### Rules for Counting Significant Figures

1. **Non-zero digits** are always significant. Example: 235 has **3** significant figures.
2. **Leading zeros** are NOT significant. Example: 0.0045 has **2** significant figures.
3. **Trailing zeros** in a decimal number are significant. Example: 3.450 has **4** significant figures.
4. **Trailing zeros in a whole number without a decimal** are NOT significant. Example: 3400 has **2** significant figures (unless written as  $3.400 \times 10^3$ ).
5. **Exact numbers** (like 100 people) have infinite significant figures.

### Rounding Off Rules

- If the digit to be removed is **less than 5**, drop it.
- If the digit is **5 or greater**, round up the previous digit.

**Example:**

- 3.146 rounded to **3 significant figures** → **3.15**
- 0.04578 rounded to **2 significant figures** → **0.046**

## Error Bounds

Error bounds define the range within which a computed value lies.

- The maximum possible deviation of an approximate value from the exact value.
- Helps in assessing the reliability of numerical computations.

### 2. Types of Error Bounds

1. **Absolute Error Bound:** The range within which the actual value lies. Example: If  $x = 5.23 \pm 0.02$ , then the absolute error bound is **0.02**.
2. **Relative Error Bound:** The absolute error divided by the true value. Example: If  $x = 5.23 \pm 0.02x$ , the relative error is:  $0.02/5.23 = 0.0038$ .
3. **Percentage Error Bound:** Relative error  $\times 100$ . Example:  $0.0038 \times 100 = 0.38\%$ .

**3. Interval Representation** If an approximate value is given as  $x \pm \delta x$ , the true value is in the range:  $(x - \delta x) \leq \text{True Value} \leq (x + \delta x)$ .

# Interpolation

- Interpolation is a method of estimating unknown values that fall within the range of a given set of known data points.
- It is used when:
  - Function is difficult or impossible to express analytically
  - Only discrete data is available (e.g., from experiments, tables)
- The goal is to construct an approximate function  $f(x)$  such that  $f(x_i)=y_i$  for known points  $(x_i, y_i)$ , and estimate  $f(x)$  for intermediate values of  $x$ .

| Feature          | Interpolation                                       | Extrapolation                                         |
|------------------|-----------------------------------------------------|-------------------------------------------------------|
| Definition       | Estimating a value <b>within</b> known data range   | Estimating a value <b>outside</b> the known range     |
| Accuracy         | Generally more accurate                             | Less accurate and more uncertain                      |
| Data requirement | Uses nearby known points                            | Projects trend beyond known data                      |
| Example          | Estimate value at $x=2.5$ using data from $x=1,2,3$ | Estimate value at $x=6$ using data from $x=1,2,...,5$ |

## Newton's Forward Interpolation (For equal intervals, near the beginning of the table)

- Used when  $x$  lies **near the beginning** of the data table.
- Assumes **equal intervals**:  $x_{i+1} - x_i = h$  (constant).
- Based on **forward difference table** using symbol  $\Delta$ .
- Approximates unknown values by using a **polynomial expansion** of the known values starting from  $y_0$ .
- Involves a variable  $u = \frac{x-x_0}{h}$  to generalize the formula.
- Effective when data points are **closely and equally spaced**.

## Newton's Backward Interpolation (For equal intervals, near the end of the table)

- Used when  $x$  lies **near the end** of the data table.
- Also assumes **equal spacing** between  $x$ -values.
- Based on the **backward difference table** using symbol  $\nabla$ .
- Uses a polynomial formed in terms of  $y_n$  (last known value).
- Introduces  $u = \frac{x-x_n}{h}$ , where  $x_n$  is the last known value.
- Useful when approximating values **after the last known data point** or very close to it.

## 1. Newton's Divided Differences

Newton's Divided Difference method is a recursive formula to construct an interpolating polynomial that passes through a given set of data points.

### ◆ Form of the Polynomial:

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

- $f[x_0]$ ,  $f[x_0, x_1]$ , etc. are divided differences computed recursively.

### ◆ Divided Difference Formula:

$$f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}$$

### ◆ Advantages:

- Works well when data points are added incrementally.
- Efficient for unequally spaced data.

## 2. Lagrange's Interpolation

Lagrange interpolation constructs a single polynomial that passes through all given points using Lagrange basis polynomials.

### ◆ Form of the Polynomial:

$$P(x) = \sum_{i=0}^n y_i \cdot L_i(x)$$

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

- $L_i(x)$  is the Lagrange basis polynomial.
- $y_i$  are the given function values at  $x_i$ .

### ◆ Advantages:

- Simple to implement.
- Useful when the entire data set is known in advance.

# CH 3: Numerical Integration

| Rule                      | Definition                                                                                | When to Use                                                          | Advantages                                                                                                                    | Disadvantages                                                                                                                              |
|---------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trapezoidal Rule</b>   | Approximates the area under a curve by dividing into straight-line segments (trapezoids). | When function is nearly linear or for rough quick approximations.    | <ul style="list-style-type: none"> <li>Simple and easy</li> <li>Needs less computation</li> </ul>                             | <ul style="list-style-type: none"> <li>Less accurate for curved functions</li> <li>Large error if function is highly non-linear</li> </ul> |
| <b>Simpson's 1/3 Rule</b> | Approximates the curve using parabolas (degree 2 polynomial) over every 2 subintervals.   | When number of intervals is <b>even</b> and function is smooth.      | <ul style="list-style-type: none"> <li>High accuracy for smooth functions</li> <li>Simple for small n</li> </ul>              | <ul style="list-style-type: none"> <li>Requires <b>even</b> number of intervals</li> <li>Not good if function changes rapidly</li> </ul>   |
| <b>Simpson's 3/8 Rule</b> | Approximates the curve using a cubic polynomial over 3 subintervals (4 points).           | When number of intervals is a multiple of 3. (i.e., 3, 6, 9,...)     | <ul style="list-style-type: none"> <li>Slightly better for cubic functions</li> <li>Handles 3 subinterval grouping</li> </ul> | <ul style="list-style-type: none"> <li>Slightly complex than 1/3 rule</li> <li>Not efficient if intervals not multiple of 3</li> </ul>     |
| <b>Weddle's Rule</b>      | Approximates using a degree 6 polynomial over 6 subintervals (7 points).                  | When <b>very high accuracy</b> is needed and nnn is a multiple of 6. | <ul style="list-style-type: none"> <li>Very high accuracy</li> <li>Good for smooth complex functions</li> </ul>               | <ul style="list-style-type: none"> <li>Tedious to apply manually</li> <li>Needs exactly 6 subintervals (or multiple of 6)</li> </ul>       |

## 🔑 Important Conditions:

| Rule               | Condition on number of intervals                 |
|--------------------|--------------------------------------------------|
| Trapezoidal        | Any number (no restriction)                      |
| Simpson's 1/3 Rule | Number of intervals must be <b>even</b>          |
| Simpson's 3/8 Rule | Number of intervals must be <b>multiple of 3</b> |
| Weddle's Rule      | Number of intervals must be <b>multiple of 6</b> |

## 📌 Quick Memory Tip:

- Straight lines → Trapezoidal (degree 1)
- Parabola fitting → Simpson's 1/3 (degree 2 but exact for degree 3)
- Cubic fitting → Simpson's 3/8
- Degree 6 fitting → Weddle's (very accurate)

# Ch 5 Solution to System of Linear Equations

Solving a system of linear equations is a fundamental problem in numerical methods. These systems can be solved using **direct methods** (which provide exact solutions under ideal conditions) or **iterative methods** (which approximate solutions and are useful for large systems). Below is a detailed explanation of the methods.

## Direct Methods

Direct methods aim to find an exact solution in a finite number of steps. These methods are efficient for smaller systems but may become computationally expensive for large matrices.

### Gauss Elimination Method

The **Gauss Elimination Method** is used to solve a system of linear equations by converting the coefficient matrix into an upper triangular form using row operations. The solution is then obtained through **back-substitution**. **Steps:**

- Convert the system into an augmented matrix.
- Use row operations to form an **upper triangular matrix** (zeroing out elements below the diagonal).
- Solve for the unknowns using **back-substitution**.

| Advantages                                          | Disadvantages                                                       |
|-----------------------------------------------------|---------------------------------------------------------------------|
| Simple and straightforward method.                  | Cannot easily handle changes in system (needs full re-computation). |
| Good for small systems.                             | Sensitive to round-off errors (especially for large systems).       |
| Always gives exact solution if no division by zero. | Computationally expensive for large matrices.                       |

### Gauss-Jordan Method

The **Gauss-Jordan Method** is an extension of Gauss Elimination where the matrix is further reduced to the **identity matrix**, eliminating both lower and upper triangular elements. **Steps:**

- Convert the system into an augmented matrix.
- Use row operations to convert the matrix into **reduced row echelon form (identity matrix)**.
- The rightmost column directly gives the values of the unknowns.

| Advantages                                                               | Disadvantages                                    |
|--------------------------------------------------------------------------|--------------------------------------------------|
| Directly gives inverse of matrix as well (along with solving $Ax = b$ ). | More computational steps than Gauss Elimination. |
| No need for back substitution step.                                      | Higher chances of round-off errors.              |
| Useful when solving multiple systems with same coefficient matrix.       | Not efficient for large systems.                 |

### LU Decomposition Method

The **LU Decomposition Method** factorizes a matrix  $A$  into two matrices:  $A=LU$ , where:

- $L$  is a **lower triangular matrix**.
- $U$  is an **upper triangular matrix**.

**Steps:**

1. Decompose  $A$  into  $L$  and  $U$ .
2. Solve  $LY=B$  using **forward substitution**.
3. Solve  $UX=Y$  using **backward substitution**.

This method is computationally efficient for solving multiple systems with the same coefficient matrix.

| Advantages                                                                               | Disadvantages                                                    |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Efficient when solving multiple systems with same matrix but different right-hand sides. | Needs additional storage for L and U matrices.                   |
| Reduces computational cost after decomposition.                                          | Decomposition fails if pivot elements are zero (needs pivoting). |
| Good for large systems.                                                                  | Initial decomposition can still be expensive.                    |

## Iterative Methods

Iterative methods are useful for solving large systems where direct methods are computationally expensive. These methods start with an initial guess and refine it through iterations.

### Gauss Jacobi Method

The **Jacobi Method** solves each variable independently in an iterative manner. **Steps:**

1. Rearrange equations so that each equation solves for a single variable.
2. Start with an initial guess.
3. Compute new values of each variable using the previous iteration's values.
4. Repeat until convergence.

| Advantages                                                       | Disadvantages                                                  |
|------------------------------------------------------------------|----------------------------------------------------------------|
| Easy to implement.                                               | Converges slowly.                                              |
| Parallel computation is possible (good for parallel processors). | Convergence not guaranteed (needs diagonally dominant matrix). |
| Suitable for simple iteration-based problems.                    | Not good for ill-conditioned systems.                          |

### Gauss-Seidel Method

The **Gauss-Seidel Method** is an improvement over Jacobi, where updated values are immediately used in the next calculations. **Steps:**

1. Rearrange equations for easy computation of each variable.
2. Start with an initial guess.
3. Compute new values using the latest available values.
4. Repeat until convergence.

| Advantages                                | Disadvantages                                                |
|-------------------------------------------|--------------------------------------------------------------|
| Generally faster convergence than Jacobi. | Convergence still not guaranteed without diagonal dominance. |
| Easy to implement with simple code.       | Cannot be parallelized easily (sequential dependency).       |
| Good for sparse and large matrices.       | May fail for badly scaled systems.                           |

**Key Difference from Jacobi:**

- **Jacobi Method:** Uses previous iteration values for all variables.
- **Gauss-Seidel Method:** Uses updated values as soon as they are available, leading to faster convergence.

## Direct Method vs Iterative Method

| Feature        | Direct Method                                     | Iterative Method                      |
|----------------|---------------------------------------------------|---------------------------------------|
| Nature         | Provides an exact solution                        | Approximates the solution iteratively |
| Computation    | High computational cost for large systems         | More efficient for large systems      |
| Error Handling | Minimal rounding errors                           | Errors propagate across iterations    |
| Examples       | Gauss Elimination, Gauss-Jordan, LU Decomposition | Jacobi, Gauss-Seidel                  |

## Gauss Elimination vs Gauss-Jordan vs LU Decomposition

| Feature     | Gauss Elimination                 | Gauss-Jordan                | LU Decomposition                                                        |
|-------------|-----------------------------------|-----------------------------|-------------------------------------------------------------------------|
| Goal        | Converts to upper triangular form | Converts to identity matrix | Factorizes matrix into <b>L</b> and <b>U</b>                            |
| Computation | Uses back-substitution            | Uses more row operations    | Efficient for solving multiple systems with the same coefficient matrix |
| Efficiency  | Faster than Gauss-Jordan          | More computational steps    | More efficient for large matrices                                       |
| Usage       | General system solving            | Inverse of a matrix         | When solving multiple equations with the same coefficient matrix        |

## Gauss-Jacobi vs Gauss-Seidel

| Feature            | Gauss-Jacobi                                     | Gauss-Seidel                                              |
|--------------------|--------------------------------------------------|-----------------------------------------------------------|
| Update Strategy    | Uses previous iteration values for all variables | Uses updated values immediately                           |
| Convergence        | Slower                                           | Faster than Jacobi                                        |
| Computational Cost | More iterations required                         | Fewer iterations needed                                   |
| Parallelization    | Can be easily parallelized                       | Harder to parallelize due to dependency on updated values |

| Method            | Best For                                     |
|-------------------|----------------------------------------------|
| Gauss Elimination | Single small system.                         |
| Gauss-Jordan      | Finding inverse + solution together.         |
| LU Decomposition  | Repeated solving with same matrix.           |
| Gauss-Jacobi      | Parallel, simple problems.                   |
| Gauss-Seidel      | Faster convergence for large sparse systems. |

# CH 5 Solution of Algebraic Equations

Throughout our discussion, we assume that

(i) the function  $f(x)$  is continuous and continuously differentiable up to sufficient number of times.

(ii)  $f(x) = 0$  has no multiple root, i.e., if  $\alpha$  is a real root of  $f(x) = 0$ , in a sufficiently small interval  $(a, b)$ , then  $f(\alpha) = 0$  and either  $f'(x) < 0$  or  $f'(x) > 0$  in  $(a, b)$ .

## Algebraic Equations

An **algebraic equation** is an equation that consists of **only algebraic expressions**, meaning it involves **polynomials with rational coefficients** and uses operations like **addition, subtraction, multiplication, division, and exponentiation with integer exponents**.

✓ **General Form:**

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 = 0$$

✓ **Examples:** Linear Equation, Quadratic Equation, Cubic Equation

## Transcendental Equations

A **transcendental equation** is an equation that involves **transcendental functions**, meaning it includes functions that are **not algebraic**, such as **exponential, logarithmic, trigonometric, or other non-polynomial functions**.

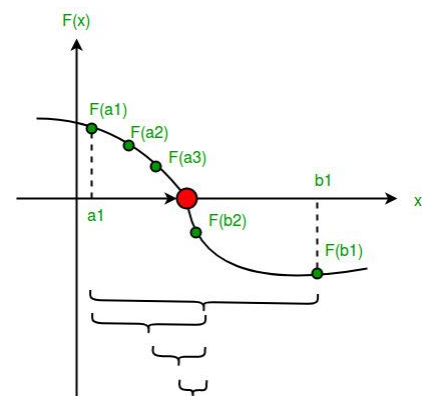
✓ **General Form:**  $f(x)=g(x)$  (Either both or  $f(x)$  or  $g(x)$  are TE)

✓ **Examples:** Exponential Equation, Logarithmic Equation, Trigonometric Equation

## Bisection Method

The **Bisection Method** is a root-finding algorithm that works by repeatedly dividing an interval  $[a, b]$  in half and selecting the subinterval where the function changes sign. It guarantees convergence if the function is continuous.

- It is a **bracketing method**, meaning it requires an interval  $[a, b]$  where  $f(a)f(b)$  have opposite signs.
- The method repeatedly halves the interval and selects the subinterval containing the root.
- It converges **linearly** to the root.
- It always works if the function is continuous and the initial interval is correctly chosen.



✓ **Advantages:**

- ✓ **Guaranteed convergence** if the function is continuous and has a sign change.
- ✓ **Simple and easy to implement.**
- ✓ **Always finds a root within the interval.**

✗ **Disadvantages:**

- ✗ **Slow convergence** (linear rate).
- ✗ **Inefficient for highly accurate results** as it requires many iterations.
- ✗ **Cannot find multiple roots** in a given interval.

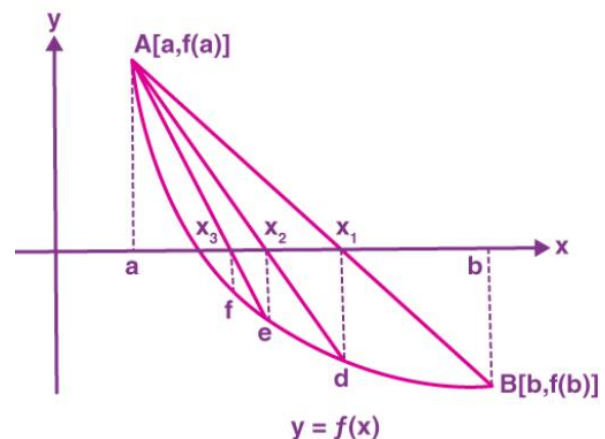
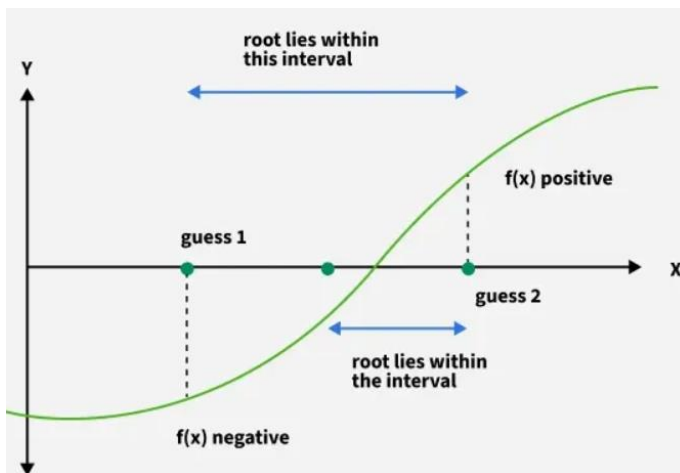
# Regula Falsi Method (False Position Method)

The Regula Falsi Method is a root-finding algorithm that improves upon Bisection by using a secant line to approximate the root, instead of halving the interval. The new root estimate is found at:

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)}$$

where  $f(a)$  and  $f(b)$  have opposite signs.

- It is also a **bracketing method**, requiring an interval  $[a,b]$  with opposite signs at endpoints.
- Instead of halving the interval, it takes a weighted average using a **secant line** to estimate the root.
- It converges **faster than the Bisection Method** but still slower than Newton-Raphson.
- The interval size does not always shrink symmetrically.



## ✓ Advantages:

- ✓ **Faster than the Bisection Method** (super-linear convergence).
- ✓ **Guaranteed to converge** if the function is continuous and the interval is chosen correctly.
- ✓ **More accurate root estimation** compared to the Bisection Method.

## ✗ Disadvantages:

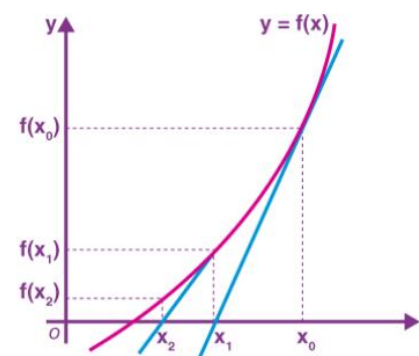
- ✗ **Convergence can be slow** in some cases when one end remains fixed for many iterations.
- ✗ **Not as fast as Newton-Raphson** for well-behaved functions.
- ✗ **Requires function evaluation at every step**, which increases computation time.

# Newton-Raphson Method

The Newton-Raphson Method is an iterative root-finding technique that uses tangent lines to approximate the root. Given an initial guess  $x_0$ , the next approximation is given by:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- It is an **open (non-bracketing) method**, meaning it starts with a single initial guess instead of an interval.
- It has **quadratic convergence**, making it much faster than both Bisection and Regula Falsi.
- Works well when the initial guess is close to the root.



✓ **Advantages:**

- ✓ **Very fast convergence** (quadratic rate).
- ✓ **Efficient for smooth, differentiable functions.**
- ✓ **Can handle multiple roots** if the function's behaviour is well understood.

✗ **Disadvantages:**

- ✗ **Requires computation of derivatives**, which may be difficult for some functions.
- ✗ **Fails if the derivative is zero or very small**, leading to division errors.
- ✗ **May diverge** if the initial guess is not chosen wisely.

| Feature                 | Bisection Method                            | Regula Falsi Method                         | Newton-Raphson Method                                  |
|-------------------------|---------------------------------------------|---------------------------------------------|--------------------------------------------------------|
| Concept                 | Halves the interval at each step.           | Uses a secant line to approximate the root. | Uses tangents to refine the root estimate.             |
| Convergence Speed       | Slow (linear convergence).                  | Faster than Bisection but may stagnate.     | Very fast (quadratic convergence).                     |
| Formula Used            | $c = \frac{a+b}{2}$                         | $c = \frac{af(b)-bf(a)}{f(b)-f(a)}$         | $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$               |
| Initial Condition       | Two values with opposite signs.             | Two values with opposite signs.             | One initial guess.                                     |
| Derivative Needed?      | ✗ No                                        | ✗ No                                        | ✓ Yes                                                  |
| Guaranteed Convergence? | ✓ Yes (if function is continuous).          | ✓ Yes (but slower in some cases).           | ✗ No (depends on good initial guess).                  |
| Efficiency              | Least efficient.                            | Moderate efficiency.                        | Most efficient when applicable.                        |
| Best for?               | Any continuous function with a sign change. | When secant approximation is effective.     | When the function is smooth & derivative is available. |